MOBILE PROGRAMMING (FLUTTER) • MIDTERM PROPOSAL

HomeServe Uzbekistan

A Cross-Platform Mobile Solution for Local Household Services & Company Management

Team Name: Flutter101 **Team Lead:** Abdujabborov Shaxriyor

Team Size: 4 Members **Course Scope:** Mobile Programming (Weeks 1–7)

1. The Problem: Real, Local, Specific

In urban centers across Uzbekistan (such as Tashkent, Samarkand, and Bukhara), residents frequently face sudden home maintenance emergencies—ranging from electrical tool malfunctions and plumbing failures to the urgent need for reliable babysitting or elderly care. Currently, finding trustworthy local masters or service companies is highly fragmented. Residents must sift through scattered Telegram channels, unverified OLX listings, or rely entirely on informal word-of-mouth recommendations.

Meeting the Four Tests for a Valid Problem:

- **Real:** Supported by our local survey of 24 residents and official Stat.uz data showing high mobile penetration (99.3%) paired with informal service booking methods (79%).
- **Local:** Tailored specifically to Uzbekistan's neighborhood (mahalla) dynamics, local service terminology (e.g., Remont, Usta), and communication habits.
- **Narrow:** Focusing explicitly on urban residents seeking urgent household help and local service providers managing daily orders.
- **Solvable with a Phone:** A Flutter mobile application directly bridges the gap with instant messaging, status filtering, and profit tracking.

2. Target Users & One-Sentence Scope

> **One-Sentence Problem Scope:** *"Urban families in Uzbekistan cannot quickly book verified solo masters or multi-service companies with transparent pricing during daily household emergencies."*

User Group A: Household Customers

Busy working parents and apartment residents needing fast repairs, cleaning, disinfection, or family care services. They require instant category browsing, transparent upfront pricing, and direct chat capability with the provider.

User Group B: Solo Masters & Companies

Independent technicians and multi-employee organizations offering services like Renovation, Gardening, and Plumbing. They need frictionless registration, service-type order filtering, and live profit monitoring.

3. Evidence Appendix (Survey & Official Statistics)

To validate that the problem is real and widespread in Uzbekistan, our team surveyed 24 local residents in Tashkent and cross-referenced official national indicators from Stat.uz.

Evidence Source	Metric / Finding	Relevance to HomeServe App
Stat.uz Official Report	99.3% Mobile phone usage	Confirms a smartphone app is the most accessible channel for households
Stat.uz Official Report	94.2% Internet penetration	Ensures reliable mobile connectivity for live order updates and chat
Team Survey (n = 24)	79.0% rely on Telegram / word-of-mouth	Proves the absence of a unified, trusted home-service directory
Team Survey (n = 24)	83.0% of masters lack digital booking tools	Demonstrates strong demand from solo masters and local companies

4. App Concept & Solution Approach

App Concept: HomeServe is a dual-mode Flutter application designed for households and service providers in Uzbekistan. Customers can quickly discover categories and book verified local help, while solo masters and multi-service companies get a dedicated business workspace to toggle availability, monitor weekly profit, filter active vs. completed orders by service category, and chat directly with clients.

5. Main User Flow (End-to-End Resolution)

The core user journey resolves a household problem in five structured steps:

Step 1: Role Selection & Onboarding — On launch, the user selects either "I'm a customer" or "I'm a master / company" to enter the tailored registration flow.

Step 2: Service Discovery & Multi-Service Setup — Customers browse categories (Electricity Repair, Appliance Repair, Disinfection, Legal Door Opening, Care); providers register their brand and select all service categories they offer.

Step 3: Request Submission — The customer submits a service request specifying their Tashkent district and offered price, which immediately appears on matching providers' Home dashboards.

Step 4: Order Acceptance & Client Chat — The provider accepts the request, moving it to the "Active" section of the Orders tab, where both parties can exchange chat messages to confirm timing and details.

Step 5: Order Completion & Profit Update — Once the repair or service is finished, the provider taps "Complete & Close Order", moving the record to history and incrementing the Total Profit counter.

6. Flutter Technical Plan (Weeks 1–7 Concepts)

Every screen in our prototype maps directly to the Flutter widgets, layouts, state management rules, and form validation techniques covered in Weeks 1 through 7:

Screen / View	Widgets & Layout	State & Navigation	Forms & Validation
1. Welcome & Auth	Scaffold, Column, Padding, ElevatedButton	StatefulWidget, Navigator.pushNamed	Form, GlobalKey<FormState>, TextFormField
2. Customer Home	ListView.builder, Row, Container, Card	Lifted state in parent AppShell	Quick booking request modal input
3. Categories Grid	GridView.count, Expanded, Icon	Pass selected category via Constructor	Search TextField with TextEditingController
4. Business Home	Column, Switch, ListView.builder	setState() for Online toggle & Profit card	Accept / Chat action callbacks
5. Service Orders	Row (Active/All toggle), DropdownButton	setState() filters list & closes active order	Chat reply TextField controller
6. Unified Profile	ListView, ListTile, CircleAvatar, Divider	Navigator.pop on Log out	Service price & working hours edit fields

State Management & Navigation Architecture

Lifting State Up

Shared application data (active service orders, provider online status, and total profit) lives in a top-level stateful root widget and is passed down via constructors and updated via callback functions.

Navigation & Routing

Top-level role switching uses named routes, while bottom tab switching uses a Scaffold with BottomNavigationBar and an indexed list of screens.

7. Core Dart Data Models

The application state is structured around three strongly-typed Dart classes with null safety:

```dart
class AppUser {
  final String id, fullName, phone, district;
  final bool isBusiness;
  AppUser({required this.id, required this.fullName, required this.phone,
required this.district, required this.isBusiness});
}

class ServiceProvider {
  final String id, brandName, description;
  final List<String> services;
  bool isOnline;
  double totalProfit;
  ServiceProvider({required this.id, required this.brandName, required
this.description, required this.services, this.isOnline = true, this.totalProfit
= 0.0});
}

class ServiceOrder {
  final String id, serviceType, clientName, district;
  double price;
  String status; // 'Active' or 'Finished'
  List<String> chatMessages;
  ServiceOrder({required this.id, required this.serviceType, required
this.clientName, required this.district, required this.price, this.status =
'Active', required this.chatMessages});
}
```

8. Out of Scope & Team Contributions

Out of Scope (Weeks 1–7)

- **Cloud Backend:** In-memory Dart lists replace Firebase or REST APIs.
- **Payments:** Payme/Click gateways are omitted; local price tracking only.
- **Live Maps SDK:** District dropdown selectors replace live GPS map pins.

Who Did What (Team Flutter101)

- **Member 1 (Lead — Abdujabborov Shaxriyor):** Problem research & presentation.
- **Member 2:** Core Dart data models.
- **Member 3:** User flow & screens.
- **Member 4:** Flutter technical plan.

AI Tools Disclosure (Per Course Rules): AI assistants (ChatGPT, Gemini) were used to help structure the proposal, refine UI wireframes, and format documentation. All team members reviewed and verified the Weeks 1–7 Flutter architecture.